The AI Reproducibility Index: Detecting Meaningful Change in Model Outputs with Deterministic Codes [TODO: title]

[TODO: authors and affiliations]

Abstract

- **Problem:** identical requests to the same model can return different outputs. Byte hashes flag every rounding difference, and task metrics (Recall@10, accuracy) average real changes away.
- **Method:** ARI turns each output vector (embedding or logits) into a deterministic code with the public SEMQ SDK, then compares codes across execution conditions. The headline score, ARI-R $= 1 - \bar{\rho}$, is the share of code positions unchanged, and it is comparable across model sizes.
- **Evidence that it is the right detector:** two pre-registered studies. At equal storage it catches every material change (48/48) and raises 0/336 false alarms, against 10–20 for float sketches and prefixes ($p \leq 0.001$). Exploratory: SimHash raises 4/336 (not significant).
- **Findings:**
  - bf16 changes the top-10 for up to 96% of queries while Recall@10 moves significantly in 2 of 48 tests;
  - $\sim$1% token change $\rightarrow$ 21–42% of greedy generations differ, on two GPU types;
  - a library upgrade silently loads 6/6 tested popular models in half precision, changing 15–100% of codes;
  - hosted embedding APIs range from bit-stable over six days to changing $\sim$88% of outputs on every call.
- **[TODO:** one-sentence implication; final abstract $\leq$ 200 words**]**

1 Introduction

- **Motivation.** Retrieval, RAG and agents assume that the same input gives the same output. In practice, outputs change with process, hardware, batch shape, precision, library version and the serving backend. **[TODO:** 2–3 concrete incidents or citations**]**
- **Why existing checks fail:**
  - *Byte hashes* change on every harmless rounding difference (100% of cross-device runs, §6).
  - *Task metrics* average changes away (§5).
  - *Float distances* need a threshold, and at small storage they false-alarm (§4).
- **Idea.** Quantize each output into a fixed, versioned code. Small rounding noise does not cross a bin edge; a material change does. Codes are small, deterministic, and show *which* coordinates moved.
- **Contributions:**
  1. **ARI protocol and score:** conditions (Table 1), a fixed-range probe, and ARI-R, a dimension-normalized score (§2).
  2. **Pre-registered validation:** two studies showing ARI separates material from benign change better than hashes and float sketches at equal storage (§4).

Preprint.

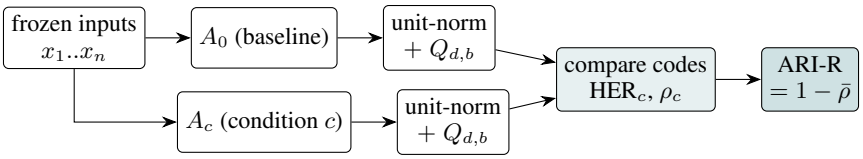


Figure 1: ARI pipeline. A baseline run and a condition run encode the same frozen inputs. Each output row is normalized and quantized by the public SEMQ SDK; codes are compared position by position. **[TODO:** add a small inset: one coordinate jittering inside a bin (no change) vs crossing an edge (change)**]**

Table 1: Conditions. The core (`proc`, `conc`, `time`) applies to any system, hosted or self-hosted; diagnostics need control of the stack. Labels follow the pre-registered rule (§4): *material* if numeric precision or quantization changes, *benign* if only scheduling, shape or device changes.

condition	varies	where	label
`same`	immediate repeat, same process (control)	all	benign
`proc`	fresh process or client	all	benign
`conc`	64-way concurrent burst vs calm pass	all	benign
`time`	$\geq$ 24 h gap, same host	all	benign
`batch`	batch size 1/8/128/512 vs 32	self-hosted	benign
`threads`	1 vs 4 CPU threads	self-hosted	benign
`mach`	CPU $\leftrightarrow$ GPU, GPU $\leftrightarrow$ GPU (fp32)	self-hosted	benign
`tf32`	TF32 matmul on vs off	self-hosted	material
`prec`	bf16 / fp16 weights vs fp32	self-hosted	material
`lib`	library default load vs explicit fp32	self-hosted	material (dtype changes)

3. **Measurements:** self-hosted embedding and chat models, GPU kernels and settings, library defaults, and five hosted APIs (§5–7).
4. **Open artifacts:** public SDK (`semq==1.0.0`), frozen inputs, signed reports, pre-registrations. **[TODO:** repo URL, DOI**]**

2 Method

System and conditions.

- System under test $A : \mathcal{X} \to \mathbb{R}^d$: an embedding model, or per-step logits under a fixed prefix (teacher-forced).
- Baseline run A_0 vs condition run A_c; each condition varies one execution axis (Table 1).
- Hosted APIs: the backend state is unobservable, so changes are *associated* with a condition, not attributed to a cause.

The probe (ARI-Canonical-v0.2).

- Normalize each row in binary64, cast to float32: $\hat{a} = a/\|a\|_2$.
- Fixed range set by the dimension alone (the SDK's `max_magnitude`): $s_d = 2/\sqrt{d}$.
- Symbol per coordinate with b magnitude bins (sign selects the half of the alphabet):

$$Q_{d,b}(\hat{a})_j \;=\; b \cdot \mathbb{1}[\hat{a}_j \geq 0] \;+\; \min\Big(\Big\lfloor \tfrac{b\,|\hat{a}_j|}{s_d} \Big\rfloor,\; b-1\Big), \qquad Q_{d,b}(\hat{a})_j \in \{0,\ldots,2b-1\}. \tag{1}$$

[TODO: confirm the sign of exact zero and the clip above s_d against the SDK spec; edges at multiples of s_d/b verified empirically**]**

- ARI-R uses $b = 2$ (2 bits per coordinate, $d/4$ bytes per vector). The logit probe uses $b = 8$.
- Deterministic and bit-exact across platforms (SDK conformance vectors); the probe adds no disagreement of its own.
- **Why a fixed range:** the v0.1 probe calibrated s on a data sample. Recalibrating on a different sample moved s by 0.3–3.6% and rewrote 60–100% of codes for *identical* vectors (§3).

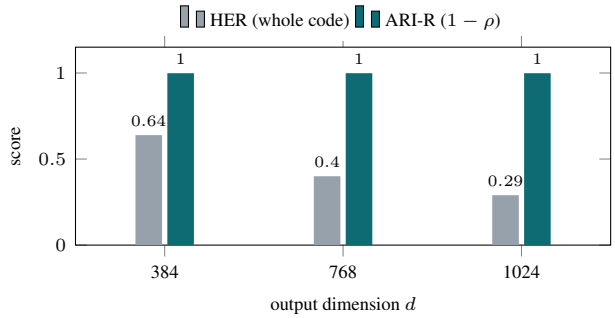


Figure 2: Same relative noise, different output sizes. Whole-code agreement penalizes larger d; the coordinate change rate does not. 1,024-d shows the midpoint of 0.26–0.32 across models.

source: `scratchpad probe_cmp (synthetic noise)` **[TODO: commit the script and its output to experiments/]**

Metrics. For inputs $i = 1..n$ and condition c, with codes $q_{0,i} = Q(\hat{A}_0(x_i))$, $q_{c,i} = Q(\hat{A}_c(x_i))$:

$$\mathrm{HER}_c = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}[q_{0,i} = q_{c,i}] \qquad \text{(whole-code exact-match rate)} \tag{2}$$

$$\rho_c = \frac{1}{n\,d}\sum_{i=1}^{n}\sum_{j=1}^{d} \mathbb{1}[q_{0,ij} \neq q_{c,ij}] \qquad \text{(coordinate change rate)} \tag{3}$$

$$\text{ARI-R} = 1 - \tfrac{1}{3}\left(\rho_{\texttt{proc}} + \rho_{\texttt{conc}} + \rho_{\texttt{time}}\right) \qquad \text{(headline; } \texttt{same} \text{ is a control)} \tag{4}$$

- 95% intervals: bootstrap over inputs, 1,000 resamples. **[TODO:** state that they cover input sampling, not serving-environment realizations**]**
- **Report both.** ρ (and ARI-R) is comparable across d; HER answers "how many outputs changed at all". Example: Voyage and OpenAI have equal ARI-R (0.99976) but HER 0.99 vs 0.93 (§7).

ARI-D (decoding).

- **Self-hosted (logits).** ARI-D-Logit-v0.2: centre each logit row (remove the shift invariance of softmax), split it into fixed 65,536-wide chunks, normalize each chunk, and apply $Q_{\cdot,8}$. Teacher-forced on the baseline tokens, so every condition sees identical prefixes. Details in App. B.
- **Hosted (text).** Rate of byte-identical completions over k repeats per prompt, plus the mean relative position of the first divergence.
- **[TODO:** decide whether ARI-D gets its own headline number or is reported per condition only**]**

3 Is ARI a valid detector?

Dimension fairness.

- Inject the same relative noise ($\varepsilon = 3 \times 10^{-3}$) into eight encoders' outputs, $d \in \{384, 768, 1024\}$.
- HER falls from 0.64 to 0.26–0.32 as d grows; ρ stays at 1.1–1.4$\times 10^{-3}$ (Fig. 2). So HER ranks models by size, not stability; ARI-R does not.

Calibration dependence (why v0.2 fixes the range).

- v0.1 probe recalibrated on FiQA vs SciFact rows: s moved 0.3–3.6%, codes changed for 60–100% of identical vectors (8 encoders).
- The fixed range has no calibration, so this failure mode is gone. **[TODO:** one line on the version pin: spec ARI-Canonical-v0.2, `semq==1.0.0`**]**

4 Selectivity: material vs benign change at matched storage

- **Question.** At equal stored bytes ($B = d/4$), which detector flags material changes and ignores benign ones?

Table 2: Selectivity at matched storage. Study 1: test half (20 material, 30 benign cells), episode rates. Study 2: all cells. p: one-sided paired sign test vs the code on Study 2 benign cells (Holm for the two pre-registered comparisons).

		Study 1		Study 2		
detector	bytes	material	benign	material	benign	p
ARI code	$d/4$	**1.000**	**0.000**	**48/48**	**0/336**	–
SHA-256	32	0.000	0.000	0/48	0/336	–
block hashes	$d/4$	0.000	0.000	0/48	0/336	–
random sketch	$d/4$	1.000	0.035	48/48	10/336	0.001
fp16 prefix	$d/4$	1.000	0.071	48/48	20/336	2×10^{-6}
full cosine†	$4d$	1.000	0.106	48/48	23/336	1×10^{-7}
SimHash*	$d/4$	1.000	0.035	48/48	4/336	0.063

†not storage-matched, secondary. *exploratory, added after both analyses, not pre-registered.

source: `experiments/selectivity-confirmatory/results/results.json`; `experiments/selectivity-benign/results/{results,exploratory_simhash}.json`

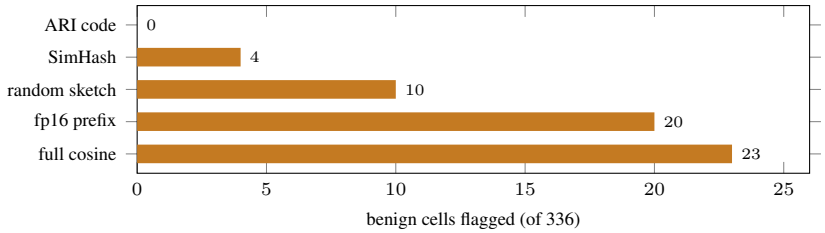


Figure 3: False alarms in Study 2. All detectors except the hashes caught 48/48 material cells. The other detectors' alarms concentrate on cross-device moves and four models (multilingual-e5-base, e5-large-v2, bge-m3, mpnet).

- **Detectors** (App. C): ARI code; SHA-256 (32 B); block hashes; fp16 prefix ($B/2$ coords); random-projection sketch ($B/4$ fp32 projections); full fp32 cosine ($4d$ B, not matched). Exploratory: SimHash ($2d$ sign bits).
- **Study 1** (pre-registered, frozen 2026-10-07): 9 encoders, 600 episodes, 90 cells. The threshold is the largest score on the calibration half's benign cells; tested on the other half.
- **Study 2** (pre-registered, frozen 2026-10-12): 25 encoders (24 analyzed), 14 benign conditions, 336 benign + 48 material cells, CPU/A10G/T4/L4. *Thresholds reused from Study 1*, so every cell is a test cell and 16 models are out-of-sample.
- **Primary test (Study 2):** paired one-sided exact sign test, code vs each matched detector on the same benign cells, Holm-adjusted.

Findings.

- Hashes cannot be selective: benign device changes alter every byte, so a zero-false-alarm threshold flags nothing (material 0/48).
- **Pre-registered claim supported:** at equal storage the code raises fewer benign alarms than the float sketch and the fp16 prefix, and catches every material cell. Code benign rate $\leq$1.1% (95% Wilson upper bound).
- Quantized codes beat continuous ones. ARI vs SimHash is *not* resolved: 4 discordant cells, all favouring ARI, $p = 0.063$. ARI's case over SimHash is design: it localizes changes, needs no shared projection matrix, and follows a versioned spec.
- **Caveats:**
  - One benign cell (MiniLM CPU vs A10G) scores exactly at the code threshold; it repeats the cell that set it.
  - 81/336 benign cells moved $\geq$1 code symbol. Selectivity comes from the threshold, not from codes never changing.
  - The code flags 53% of material *inputs*: it is a run-level detector.

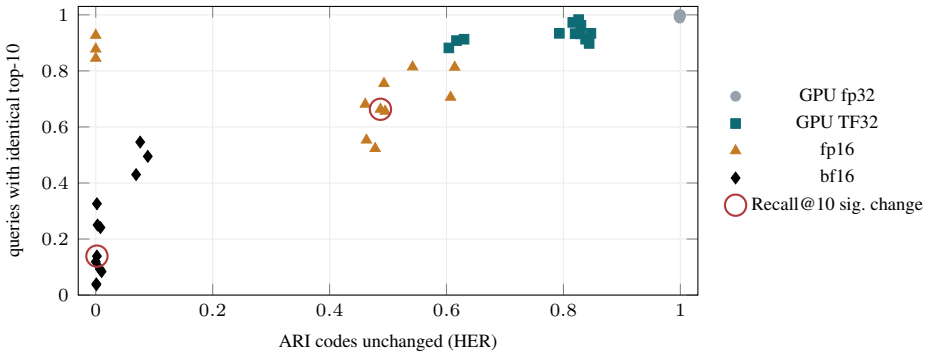


Figure 4: 48 retrieval runs (12 per setting). Codes and per-query rankings move together; Recall@10 changes significantly in only 2 runs (circled: bf16, NFCorpus/bge-large, -0.2 pts; fp16, SciFact/EmbeddingGemma 2, -1.5 pts). Under bf16, as few as 4% of queries keep their top-10.

source: `s3 ari-v02-reruns/casestudies{,_rest}/regime_matrix_*_b2.json` [TODO: commit these JSONs to the repo]

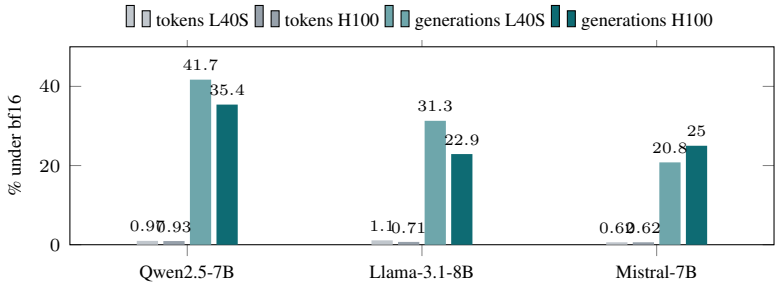


Figure 5: Under bf16, $\leq$1.1% of teacher-forced tokens change, but 21–42% of complete greedy generations differ, on both GPUs.

source: `s3 ari-v02-reruns/{l40s,l40s_retry,h100_extra}/decoding_matrix_*_gpu.json`

- [**TODO:** one sentence: why sub-threshold invariance is the mechanism (bin edges vs jitter magnitude); maybe a histogram of per-input ρ for benign vs material]

5 Changes that task metrics miss

Retrieval. 3 BEIR corpora (SciFact, NFCorpus, ArguAna) $\times$ 4 encoders (MiniLM, bge-large, e5-large, EmbeddingGemma 2); one setting changed at a time on an A10G; reference fp32 on CPU.

- Recall@10 is an average over relevance judgments; a reordered top-10 keeps the average when the relevant documents stay in. [**TODO:** 1 sentence why; cite]
- MiniLM changes *every* code under fp16 on all three corpora (HER 0.000), while larger models keep $\sim$0.5. [**TODO:** explain before submission: fp16 range in a small model? check activations]

Decoding. Three 7–8B instruct models, 48 prompts, 48 greedy tokens; L40S and H100.

- Compounding: one flipped token early changes the rest of the generation. [**TODO:** plot first-divergence position vs top-2 margin; data in decoding JSONs (median margin at flipped tokens 0.04–0.06 vs 3.2–4.2 overall)]

6 Where changes come from

Same kernel, same output. Every self-hosted repeat that keeps the kernel fixed is bit-identical: CPU (9 encoders, including at a 24–30 h gap), and A10G and H100 in a fresh process with TF32 on or off and determinism on or off (8/8 encoders each).

Table 3: ARI-D logit codes see changes before the text does. Ranges over 3 models × 2 GPUs (L40S, H100), vs fp32.

condition	logit codes unchanged	generations identical
fresh process, 1 thread	1.000	48/48
padded batch of 4	0.25–0.78	48/48
TF32 on	0.000	48/48
fp16 weights	0.000	43–46/48
bf16 weights	0.000	28–38/48

Table 4: Benign kernel changes in Study 2 (24 encoders, 1,000 inputs per cell, fp32). Raw bytes change for nearly every input under GPU batch or device changes; codes move by at most ∼2 positions in 384,000.

change	inputs w/ byte change	cells w/ byte change	cells w/ code change	max ρ
fresh process (CPU, GPU)	0.000	0/24	0/24	0
1 vs 4 CPU threads	0.005	15/24	0/24	0
CPU batch 8 / 128	0.14 / 0.08	22 / 24	0 / 2	2.6×10^{-6}
GPU batch 1 / 8 / 128 / 512	0.96–1.00	24/24	7–12	5.2×10^{-6}
CPU ↔ A10G / T4 / L4	1.000	24/24	8–11	5.2×10^{-6}
A10G ↔ T4 / L4	0.92–1.00	24/24	4–11	5.2×10^{-6}

source: `experiments/selectivity-benign/results/results.json (per-cell scores)`

Different kernel: bytes change, codes almost never.

Settings that change outputs.

- **Precision** dominates: a fresh bf16 load keeps 0.1–7.3% of codes on CPU (9 encoders); fp16/bf16 on GPU in Fig. 4.
- **TF32** changes 15–40% of codes vs off (A10G), with no change in Recall@10 or any generation. It is a configuration difference, not nondeterminism. PyTorch's default is off for matmul.
- **TF32 × batch** (H100): batch changes move codes for some encoders only when TF32 is on (Table 5).
- **Library defaults** (Fig. 6): 35 of the top-200 HF embedding models declare half precision. transformers 5 loads it by default (v4 loaded fp32), changing 15–100% of codes. One model (gte-multilingual-base) no longer runs.
- **Broken outputs:** EmbeddingGemma 2 in fp16 emits NaN vectors for 25/5,183 SciFact docs (0.48%; identical in two runs), 9/3,633 NFCorpus and 1/8,674 ArguAna. The SciFact case gives the largest Recall@10 drop observed (-1.5 pts).

7 Hosted systems

- Stability is per product, not per provider: Gemini's embedding API is bit-stable over six days; Gemini chat (below) is not.
- Cohere: ∼88% of outputs change on every call, but no more after six days: constant noise, not drift.
- OpenAI: stable per request, but batching changes 94% of codes. **[TODO:** practitioner note: batch shape is part of the contract**]**
- **Hosted chat (ARI-D, text):** gemini-3.6-flash, temperature 0, fixed seed, 100 prompts × 8. Identical pairs: 0.626 within day 1, 0.564 within day 2, 0.594 [0.559, 0.633] across 29.8 h. No time effect beyond within-batch variation.
- **[TODO:** OpenRouter gateway control and Mistral chat (blocked: key, quota); decide whether the gateway case stays**]**
- **Deviation from v0.1:** Voyage `conc` 0.209 → 0.994. The probe changed too, so we cannot attribute the difference. **[TODO:** keep in appendix only?**]**

Table 5: H100 batch invariance: codes unchanged vs batch 32 under the same settings, 9 encoders, 1,000 inputs. Most encoders stay ≥ 0.996 with TF32 on; mpnet, nomic and EmbeddingGemma 2 set the minima.

	same batch, new process	b=1	b=8	b=128	b=512
TF32 off (det. on/off)	1.000	0.999–1.000	0.999–1.000	0.999–1.000	0.999–1.000
TF32 on	1.000	0.792–1.000	0.830–1.000	0.864–1.000	0.825–1.000

source: `s3 ari-v02-reruns/h100/batch{,_gemma}/summary.json`

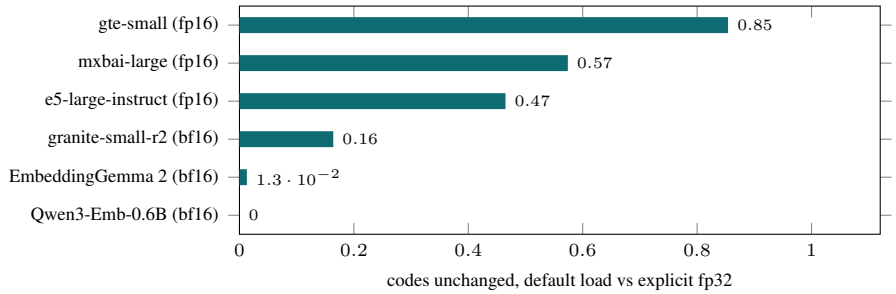


Figure 6: The `lib` condition on an H100 with transformers 5.19: the same user code, `SentenceTransformer(model_id)`, now loads the checkpoint's declared half precision. gte-multilingual-base fails to run (not shown). EmbeddingGemma 2 requires transformers 5, so for it this is the default rather than a change.

source: `s3 ari-v02-reruns/h100_extra/lib_fix/*.json; HF scan hf_dtype_scan.json` [TODO: commit both]

8 Discussion

Recommendations for practitioners.

- Pin precision explicitly (`torch_dtype`) and assert it after loading; do not rely on library defaults.
- Treat TF32, batch shape and hardware as part of the deployment contract; record them with the index.
- Store ARI codes ($d/4$ bytes) alongside embeddings; compare after every upgrade. A hash will always differ across hardware; a task metric will usually say "no change".
- For hosted APIs, measure `same` first: some services are noisy on every call.

What ARI is not.

- Not a quality metric: a code change says outputs moved, not that they got worse.
- Not a per-output detector: it reliably flags runs, and about half the changed inputs within a run.
- Not causal on hosted systems.

Limitations.

- One input set (1,000 frozen texts, English-heavy). **[TODO:** language mix]
- The threshold was set on 9 encoders and carried to 16 more; zero margin on one benign cell.
- Hosted results are a snapshot; services change without notice.
- ARI vs SimHash unresolved; a larger pre-registered study is future work.
- **Correction:** v0.1 reported that TF32-on runs disagree across processes. That came from a harness bug (the worker process did not apply TF32). Corrected runs repeat exactly (App. E).

9 Related work

- Nondeterminism in deep learning and batch invariance. **[TODO:** cite: PyTorch reproducibility docs; batch-invariant kernels work; floating-point non-associativity]
- LLM serving nondeterminism and temperature-0 variance. **[TODO:** cite]
- Hashing and sketching: SHA-256, LSH/SimHash (Charikar 2002), random projections. **[TODO:** cite]

Table 6: Hosted embedding APIs, 1,000 frozen inputs. HER per condition; `time` 148–150 h after the baseline; `conc`: 64-way burst. `batch` (diagnostic): same texts sent batched vs one per request.

API	`same`	`proc`	`conc`	`time`	`batch`	ARI-R
gemini-embedding-001	1.000	1.000	1.000	1.000	–	1.000000
voyage-4-large	0.997	0.990	0.994	0.996	0.996	0.999758
text-embedding-3-large	0.934	0.957	0.943	0.925	0.063	0.999760
embed-v4.0	0.116	0.124	0.165	0.123	0.058	0.989757
mistral-embed	**[TODO:** rerun; times out under the 16-way burst]					

source: `scratch api_panel/submissions/*.json` `[TODO: commit signed reports]`

- Embedding quantization and compact codes: binary embeddings, PQ, quantized hyperdimensional representations (qFHRR). **[TODO:** cite; one sentence each]
- ML reproducibility checklists and benchmarks. **[TODO:** cite]

10 Conclusion

- **[TODO:** 3 sentences: deterministic codes detect what hashes over-flag and metrics miss; validated by pre-registration; changes come from configuration, not randomness, and from silent defaults]

Reproducibility statement. **[TODO:** SDK version and pin; frozen input hash; pre-registration commits (565aa57, 72f1247); signed reports (KMS key id); hardware list; S3 → repo migration of raw results]

A Changes from v0.1

area	v0.1	v0.2
SDK	private pre-release build	public `semq==1.0.0`, pinned
probe range	calibrated on a sample (99th pct.)	fixed $2/\sqrt{d}$
headline score	ARI-HER (+ noise-scale $\hat{\sigma}$)	ARI-R $= 1 - \bar{\rho}$; HER reported alongside
logit probe	embedding probe on raw logits	centred, 65,536-wide chunks, 8 bins
precision	library default	set and asserted per capture
`time` (self-hosted)	not a true 24 h gap	enforced $\geq$24 h, same host
models	8 encoders	+ EmbeddingGemma 2; 25 in Study 2
evidence	exploratory	two pre-registered studies
retrieval	SciFact, 1 encoder, 8 bins	3 corpora $\times$ 4 encoders, 2 bins
TF32 claim	"breaks cross-process repeatability"	bug; corrected (App. E)
removed	ARI-E, 15-hop walk, regions modules	out of scope or unsourced

B Logit probe details

- Centre: $\ell \leftarrow \ell - \frac{1}{V}\sum_v \ell_v$. Chunk width $W = 65{,}536$ (SDK maximum dimension); normalize each chunk; $Q_{W,8}$.
- Exploratory comparison on Qwen2.5-0.5B: catches 82.5% of GPU-fp32 changes (vs 71.3% for the v0.1 probe). A constant shift of $+3$ changes 1.7% of codes (vs 100%). Spearman 0.84–0.86 with centred drift. **[TODO:** commit logit_cmp outputs]

C Detector definitions

- fp16 prefix: first $B/2$ coordinates in fp16; statistic $\frac{1}{k}\sum_j |a_j - a'_j|$.
- Sketch: $P \in \mathbb{R}^{d\times B/4}$, $P_{jk} \sim \mathcal{N}(0, 1/k)$, seed d; statistic $\|(a - a')P\|_2$.
- SimHash: $R \in \mathbb{R}^{d\times 2d}$ Gaussian, seed $10^4 + d$; statistic: fraction of $\mathrm{sign}(aR)$ bits flipped.
- Block hashes: $B/32$ SHA-256 digests over contiguous coordinate blocks; statistic: fraction of blocks changed.

D Pre-registrations and deviations

- Study 1: frozen `565aa57`. Deviation: CPU host out of memory at 2 parallel captures; resumed at 1; 2 captures repeated. Note added after results: the code's benign interval [0,0] understates uncertainty (Wilson upper bound 0.114).
- Study 2: frozen `72f1247`. Amendment before capture: GPU regions. Qwen3-Embedding-0.6B dropped (batch 512 out of memory on A10G), as pre-registered.

E Errata: TF32 cross-process measurement

- Bug: the `proc` worker in `gpu_capture.py` did not apply TF32 or determinism flags, so "TF32 on, new process" compared on vs off. Present since the first harness commit; fixed in `a617e48`.
- Corrected: A10G and H100, 8 encoders, TF32 on/off $\times$ determinism on/off: `proc` HER 1.000 in all 32 cells per GPU.
- Superseded claims: v0.1 "TF32 alone breaks GPU reproducibility" and "the deterministic flag had no effect". **[TODO:** list exact v0.1 sentences**]**

F Hardware and software

- CPU: AWS c7i.2xlarge. GPUs: A10G (g5), T4 (g4dn), L4 (g6), L40S (g6e), H100 80GB (p5.4xlarge capacity block).
- Python 3.12; torch 2.6.0 (cu124), transformers 4.57.6, sentence-transformers 4.1.0. EmbeddingGemma 2: torch 2.14.1, transformers 5.19.0, sentence-transformers 6.1.0.
- **[TODO:** note: on Hopper with the DLAMI, system CUDA 13.2 libraries on LD_LIBRARY_PATH break torch 2.14.1's cuDNN; we unset it**]**